

UML 2.5 ACTIVITY PROCESS GUIDE

AeroBook Flight Platform

Complete UML Activity Diagram & Workflow Specification

Comprehensive operational guide detailing end-to-end flight booking, dynamic pricing, seat concurrency control, and airport check-in across 6 microservice swimlanes.

WORKFLOW MODELING PARAMETERS

Standard Notation:

UML 2.5 Activity Model

Primary Process Flow:

Booking & Check-In Lifecycle

Visual Theme:

Black Canvas & White Marker

Partitioning Model:

6 Swimlanes (Service Scope)

Concurrency Control:

Pessimistic Row-Level Locking

Resilience Model:

Saga Compensation + Feign

Executive Summary

In distributed enterprise software systems, a UML Activity Diagram models the dynamic operational workflow, decision logic, data flows, and parallel synchronization across discrete system components. Unlike sequence diagrams which focus on linear message chronologies, activity diagrams excel at displaying conditional decision branchings, fork/join concurrency, and organizational responsibilities through swimlanes.

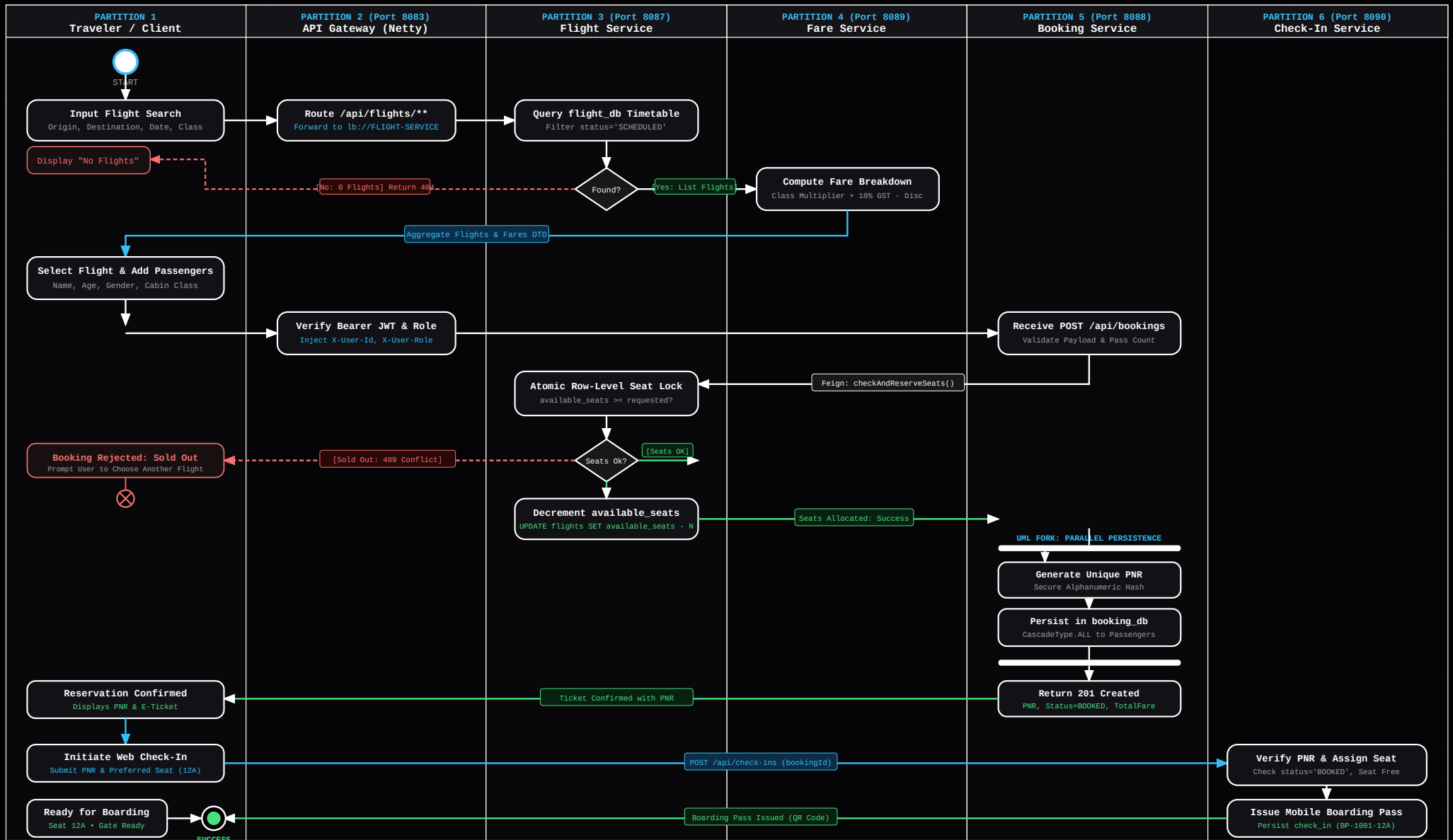
The AeroBook Activity Diagram documents the primary end-to-end customer journey from flight discovery, dynamic pricing calculation, atomic seat inventory locking, cryptographic PNR generation, to web check-in and QR boarding pass issuance. By segmenting the workflow into 6 swimlanes (Traveler, API Gateway, Flight Service, Fare Service, Booking Service, Check-In Service), this engineering specification clarifies exactly which service executes each action, how race conditions are prevented during flash sales, and how the system handles exceptions without distributed database deadlocks.

DOCUMENT TABLE OF CONTENTS

Page 2	High-Resolution UML Activity Diagram (Full Landscape Vector) Complete 6-swimlane workflow in black canvas & white marker theme
Page 3	Section 1: Fundamentals of UML 2.5 Activity Diagrams Initial/final nodes, action states, decisions, fork/join bars, and swimlane partitions
Page 4	Section 2: Flight Discovery, Ingress Routing & Dynamic Pricing Phase 1 & 2: Search criteria, Netty gateway routing, and fare pricing breakdown
Page 5	Section 3: Atomic Seat Allocation & Booking Orchestration Phase 3 & 4: JWT validation, row-level locking, PNR creation, and cascade persistence
Page 6	Section 4: Airport Web Check-In & Digital Boarding Passes Phase 5: Seat selection, passenger manifest verification, and QR code delivery
Page 7	Section 5: Exception Handling, Concurrency & Compensations Race condition prevention during flash sales, Saga rollbacks, and circuit breakers
Page 8	Section 6: Swimlane Traceability Matrix & Technical Defense FAQ Partition-to-service mapping and 4 critical technical review interview questions

AEROBOOK PLATFORM - SYSTEM UML ACTIVITY DIAGRAM

End-to-End Flight Booking, Dynamic Pricing, Concurrency Control & Airport Web Check-In • 6 Swimlanes • UML 2.5 Standard



UML 2.5 ACTIVITY DIAGRAM NOTATION LEGEND & CONCURRENCY PROTOCOLS

AeroBook Microservices Orchestration • ISO/IEC 19505 Standard

UML 2.5 CONTROL NODES:

- Initial Node: Workflow initiation trigger.
- Activity Final Node: Complete end of activity flow.
- Decision / Merge: Conditional branching [Guard conditions].

SYNCHRONIZATION & ACTIONS:

- Action State: Unit of executable computation.
- Fork / Join Bar: Concurrent execution & synchronization.
- Flow Final Node: Terminates single alternative branch.

CONCURRENCY & RESILIENCE RULES:

- Row-level lock on MySQL flights row.
- Feign client rollback increments seats if payment fails.
- Unique boarding_pass_number constraint prevents double-booking.
- Synchronous Feign + Circuit Breaker fallback timeouts.

1. Fundamentals: UML 2.5 Activity Diagrams

Visualizing Dynamic Process Control, Concurrency, and Service Partitions

An Activity Diagram specifies the dynamic behavioral logic of an operation or business process. It illustrates the step-by-step sequence of action states, showing how control flows from action to action, how branches diverge based on guard conditions, and where concurrent processing occurs.

Initial Node (Filled Circle) & Activity Final Node (Bullseye Circle)

The Initial Node marks the starting point of the activity flow. The Activity Final Node marks the termination of the entire activity; upon reaching it, all ongoing execution threads within the diagram cease.

Action State (Rounded Rectangle)

Represents an atomic, non-interruptible unit of execution within the workflow (e.g. "Query flight_db", "Compute Fare Breakdown", "Verify Bearer JWT").

Decision & Merge Nodes (Diamonds)

A Decision Diamond evaluates guard conditions (e.g. [Seats Available?]) and directs control along exactly one outgoing edge. A Merge Diamond unites multiple alternative paths without synchronization.

Fork & Join Synchronization Bars (Thick Horizontal/Vertical Bars)

A Fork bar splits a single control flow into two or more concurrent execution paths (e.g. PNR generation and fare summation). A Join bar synchronizes multiple parallel flows, waiting for all incoming paths to finish before continuing.

Swimlanes / Partitions (Vertical Columns)

Partitions assign organizational responsibility. In AeroBook, each vertical swimlane represents a distinct microservice boundary, visually demonstrating which microservice executes each step.

Control Flow vs Object/Data Flow

How Information Travels Across Microservices

Control Flow (Solid White Directed Arrows)

Dictates the sequential passage of execution authority from one completed action state to the next. For example, once the flight query finishes, control transitions directly to the flight evaluation decision node.

Object / Data Flow (Cyan Dashed Annotations)

Represents the transmission of structured data objects (DTOs) between microservices. In AeroBook, FlightSearchCriteria, FareResponseDTO, BookingRequestDTO, and BoardingPassDTO carry state across network boundaries.

Exception & Rejection Flow (Red Dashed Arrows)

Defines failure recovery pathways. If a flight has 0 available seats, control breaks out of the booking flow and triggers an HTTP 409 Conflict rejection, terminating that specific branch without affecting other platform users.

2. Flight Discovery & Dynamic Fare Pricing

Phase 1 & 2: Client Search, Gateway Ingress Routing, and Yield Pricing

Step 1: Input Search Criteria & Ingress Dispatch

Swimlanes: Traveler UX -> API Gateway (Port 8083) -> Flight Service (Port 8087)

- * Traveler Action: Enters origin (DEL), destination (BOM), travel date (2026-10-15), and cabin preference (ECONOMY).
- * Gateway Routing: api-gateway intercepts GET /api/flights/search, matches the route predicate, and uses Netty non-blocking I/O to forward the request to lb://FLIGHT-SERVICE.
- * Security Policy: Flight schedule queries are public; no Bearer JWT authorization is demanded at this stage.

Step 2: Timetable Query & Decision Node 1

Swimlane: Flight Service (Port 8087) | Database: flight_db

- * Database Operation: flight-service executes a filtered SELECT on the flights table matching origin, destination, and date with status="SCHEDULED".
- * Decision Diamond 1 [Flights Found?]:
 - Branch [No (0 results)]: Emits HTTP 404 Not Found. Client displays an informative "No Flights Available" alert, directing the user to adjust dates.
 - Branch [Yes (>= 1 flight)]: Collects flight IDs and dispatches a synchronous internal call to fare-service for yield valuation.

Step 3: Dynamic Multi-Tier Pricing Computation

Swimlane: Fare Service (Port 8089) | Database: fare_db

- * Tariff Rules: fare-service looks up fare records linked to the target flight ID in fare_db.
- * Pricing Formula: Total Fare = (Base Fare * Cabin Multiplier) + Taxes (18% GST) - Promotional Discount.
- * Cabin Multipliers: Economy = 1.0x, Business = 2.5x, First Class = 4.0x.
- * DTO Aggregation: Merges schedule timetables with live fare pricing and streams the composite catalog to the client.

PERFORMANCE NOTE: Why Decouple Flight Query from Fare Calculation?

By separating timetable lookups from dynamic pricing, flight schedules can be aggressively cached (e.g. in Redis). Meanwhile, fare-service can adjust prices in real-time based on seat scarcity without invalidating timetable caches.

3. Atomic Seat Allocation & Booking Orchestration

Phase 3 & 4: Security Verification, Concurrency Control, and Cascade Persistence

Step 4: Reservation Command & Gateway JWT Verification

Swimlanes: Traveler UX -> API Gateway (Port 8083) -> Booking Service (Port 8088)

- * Client Action: Traveler selects flight, specifies passenger list, and submits POST /api/bookings.
- * Gateway Filter: JwtAuthenticationFilter intercepts the Bearer token, validates HMAC-SHA256 signature, and extracts user identity.
- * Header Propagation: Gateway decorates the downstream request with X-User-Id and X-User-Role headers.
- * Ingestion: booking-service receives the verified payload and orchestrates the reservation transaction.

Step 5: Atomic Seat Check & Decision Node 2 (Concurrency Control)

Swimlane: Flight Service (Port 8087) | Row-Level Locking in flight_db

- * Feign Invocation: booking-service calls flight-service via OpenFeign: reserveSeats(flightId, passengerCount).
- * Concurrency Isolation: flight-service acquires a row-level lock on the flight record in MySQL InnoDB.
- * Decision Diamond 2 [Seats Available?]:
 - Branch [No (Sold Out)]: Releases lock and returns HTTP 409 Conflict. Booking service aborts and alerts customer.
 - Branch [Yes (Available)]: Executes atomic decrement: UPDATE flights SET available_seats = available_seats - ? WHERE id = ?.
- * Result: Prevents phantom seat allocation and double-booking during concurrent booking spikes.

Step 6: Fork Bar, PNR Generation & Cascade Persistence

Swimlane: Booking Service (Port 8088) | Database: booking_db

- * UML Fork Bar: Splits processing into parallel sub-tasks:
 - Branch A: Generates a cryptographically unique 6-character PNR code (e.g. PNR-AB789).
 - Branch B: Computes final ticket fare and attaches passenger records.
- * Database Insertion: Saves parent Booking entity in booking_db. CascadeType.ALL automatically inserts all child Passenger records.
- * UML Join Bar: Synchronizes persistence completion and emits HTTP 201 Created with PNR receipt to client.

ACID COMPOSITION GUARANTEE: Booking *-- Passenger

Because passengers are mapped via CascadeType.ALL and orphanRemoval=true, any failure during passenger persistence automatically rolls back the entire booking transaction, guaranteeing zero partial or orphaned database records.

4. Airport Web Check-In & Digital Boarding Passes

Phase 5: Passenger Verification, Seat Matrix Locking, and QR Code Issuance

Step 7: Web Check-In Request & Reservation Validation

Swimlanes: Traveler UX -> Check-In Service (Port 8090) -> Booking Service (Port 8088)

- * Traveler Trigger: Customer accesses digital check-in within 24 hours of flight departure, submitting PNR and preferred seat.
- * Cross-Service Verification: check-in-service invokes booking-service via OpenFeign to verify booking existence.
- * Validation Rules:
 - Booking status must equal BOOKED (cannot check in for CANCELLED flights).
 - Passenger legal name must match reservation record exactly.

Step 8: Cabin Seat Matrix Locking & Boarding Pass Issuance

Swimlane: Check-In Service (Port 8090) | Database: checkin_db

- * Seat Matrix Check: Inspects assigned seat coordinate (e.g. 12A). Verifies seat has not already been claimed by another passenger.
- * Boarding Pass Generation: Generates an alphanumeric reference code (e.g. BP-1001-12A).
- * Persistence: Saves check_in record in checkin_db with timestamp and status=CHECKED_IN.
- * QR Encoding: Assembles digital boarding barcode payload containing passenger name, flight code, seat number, and gate security hash.

Step 9: Mobile Delivery & Activity Final Node

Swimlane: Traveler / Client UX | Workflow Finalization

- * Client Receipt: Traveler receives responsive digital boarding pass with embedded scannable QR code on mobile screen.
- * Airport Security Ready: Boarding pass is saved offline in local storage for airport TSA / CISF gate scanners.
- * Activity Final Node: Workflow terminates successfully with status COMPLETE.

IDEMPOTENCY GUARANTEE: Zero Duplicate Passes

checkin_db enforces a UNIQUE constraint on boarding_pass_number. If a traveler clicks the check-in button twice in rapid succession, the second request is safely recognized as idempotent and returns the existing boarding pass.

5. Exception Handling, Concurrency & Compensations

Distributed Failure Recovery, Race Condition Prevention, and Saga Compensations

Race Condition Prevention during Concurrent Seat Booking

When thousands of users attempt to book the final 5 seats on a flight during a flash promotion, naive implementations suffer from lost updates and overselling. AeroBook prevents this by executing seat decrements inside flight-service using an atomic conditional update: UPDATE flights SET available_seats = available_seats - ? WHERE id = ? AND available_seats >= ?. If available_seats < requested count, the row update count is 0, triggering an immediate HTTP 409 Conflict rollback without corrupting inventory.

Saga Compensating Action on Booking Cancellation

When a customer cancels a confirmed reservation via PUT /api/bookings/{id}/cancel, booking-service marks the record as CANCELLED in booking_db and dispatches a compensating HTTP call to flight-service: PUT /api/flights/{id}/release-seats?count=N. Flight service increments available_seats back up in flight_db, returning inventory to the market without requiring distributed 2PC locks.

Resilience4j Circuit Breaker Fallbacks

If fare-service or flight-service experiences degraded performance or downtime, OpenFeign client calls are protected by Resilience4j Circuit Breakers. If response time exceeds 5000ms or error rate exceeds 50%, the circuit transitions to OPEN, immediately returning a graceful fallback response rather than blocking Gateway threads.

Expired JWT Token Handling at API Gateway

If a traveler submits a booking request after their 24-hour access token has expired, the Gateway JwtAuthenticationFilter halts the request before it reaches downstream microservices, issuing HTTP 401 Unauthorized with a WWW-Authenticate header instructing the client to rotate tokens via POST /api/auth/refresh-token.

6. Swimlane Traceability Matrix & Defense FAQ

Action-to-Service Mapping & Critical Technical Review Defense

SWIMLANE ACTION TRACEABILITY MATRIX

Input Flight Search	Traveler UX	GET /api/flights/search	Public Client Action
Route Ingress & JWT Filter	API Gateway	Filter: JwtAuthenticationFilter	Security & Route Proxy
Query Timetable & Seats	Flight Service	FlightRepository.findBySourceAndDest()	Read from flight_db
Compute Yield Pricing	Fare Service	FareService.calculateFare()	Read from fare_db
Atomic Seat Lock	Flight Service	PUT /api/flights/{id}/reserve-seats	Pessimistic Row Lock
PNR & Cascade Persist	Booking Service	POST /api/bookings	Write to booking_db
Web Check-In & Boarding	Check-In Service	POST /api/check-ins	Write to checkin_db

Technical Review & Viva Defense Questions

Direct Answers to Critical Process Architecture Challenges

Q1: What is the main difference between an Activity Diagram and a Sequence Diagram?

A Sequence Diagram illustrates message exchanges between objects ordered strictly by time (ideal for RPC and Feign call traces). An Activity Diagram visualizes the overarching operational control flow, conditional branching (decisions), parallel execution (fork/join), and organizational boundaries (swimlanes), making it ideal for business process modeling.

Q2: How does the system handle an unexpected server crash halfway through a booking?

If booking-service crashes after seat decrement but before committing the booking record, the flight-service reservation expires via an automatic TTL or is reverted when the customer retries the booking. Because the database-per-service pattern is used, no distributed locks remain hung in memory.

Q3: Why are Fork and Join synchronization bars used during booking persistence?

Fork and Join represent concurrency. During reservation processing, generating the PNR hash and calculating line-item tax totals can execute concurrently before synchronizing at the Join bar to commit the atomic booking transaction.

Q4: How does the activity flow prevent duplicate boarding pass issuance during web check-in?

check-in-service validates that the reservation status is BOOKED and checks its seat allocation matrix. Furthermore, checkin_db enforces a database-level UNIQUE constraint on boarding_pass_number, guaranteeing that exactly one boarding pass exists per passenger.

